

# File permissions in Linux

## Project description:

### Project Overview

In this project, I performed a comprehensive security audit and permission remediation across critical organizational files and directories in a Linux environment. Operating under the **Principle of Least Privilege (PoLP)**, I analyzed existing file system permissions, identified security vulnerabilities, and reconfigured access control settings to align with organizational security compliance standards.

### Scenario & Objectives

As part of an internal system audit, the security team identified several compliance risks involving file access across sensitive project repositories. My objectives were to:

- **Audit Directory Contents:** Systematically inspect files, subdirectories, and hidden attributes to evaluate existing access rights.
- **Revoke Excessive Permissions:** Restrict unauthorized write access for external/other users on sensitive operational files.
- **Protect Archived Assets:** Reconfigure permissions on archived data to prevent accidental or unauthorized modification while retaining read accessibility.
- **Restrict Directory Traversal:** Enforce execution restrictions on sensitive directories to prevent unauthorized users from traversing or accessing restricted project drafts.

### Technical Execution & Methodology

#### 1. Access Control Analysis & Auditing

Using Linux CLI tools (`ls -la`), I audited the 10-character POSIX permission strings for all files and subdirectories. This enabled me to map explicit access rights across three distinct user categories: **User (Owner)**, **Group**, and **Other**.

#### 2. Security Remediation via `chmod`

- **Securing Operational Files:** Applied symbolic `chmod` operations (`chmod o-w`) to strip write access from the "others" category, mitigating the risk of unauthorized data tampering.
- **Hardening Archived Records:** Applied multi-attribute permission updates (`chmod u-w,g-w,g+r`) to lock archived records into a strict read-only state across all user levels (`-r--r--r--`).
- **Enforcing Traversal Boundaries:** Revoked group execute rights (`chmod g-x`) on restricted subdirectories (`drafts`) to ensure only designated owners could traverse and execute commands within the directory structure.

### Key Outcomes & Impact

- **Enhanced Data Integrity:** Eliminated unauthorized write privileges across sensitive project files, securing the system against unauthorized modifications.

- **Compliance Alignment:** Successfully brought directory permission structures into full alignment with organizational access control policies.
- **Verifiable Security Baseline:** Validated all permission modifications post-remediation, ensuring robust access restrictions without disrupting standard user workflows.

## Check file and directory details

The following code demonstrates how I used Linux commands to determine the existing permissions set for a specific directory in the file system.

```
(swart@Kali)-[~/Projects]
$ la -ls
total 4
4 drwxrwxr-x 2 swart swart 4096 Jul 30 13:22 drafts
0 -rw-rw-r-- 1 swart swart 0 Jul 30 13:27 projects_k.txt
0 -rw-rw-r-- 1 swart swart 0 Jul 30 13:27 projects_m.txt
0 -rw-rw-r-- 1 swart swart 0 Jul 30 13:27 projects_r.txt
0 -rw-rw-r-- 1 swart swart 0 Jul 30 13:27 projects_t.txt
0 -rw-rw-r-- 1 swart swart 0 Jul 30 13:26 projects_x.txt
0 -rw-rw-r-- 1 swart swart 0 Jul 24 13:59 vulnerabilities.txt

(swart@Kali)-[~/Projects]
$
```

Here is a rewrite and explanation of your screenshot, corrected to match the actual data shown in the image.

### Rewritten Explanation:

The first line of the screenshot shows the command I executed: `la -ls`. The subsequent lines are the generated output. This lists the contents of my current `Projects` directory.

I chose the `la` command (often an alias or variant of `ls`) with the `-ls` options to get a list that is **sorted by size** (the `-S` part of that combination) in a long listing format. The output indicates that there is **one directory named drafts** and **six other project files**, including a file named `vulnerabilities.txt`. The 10-character string in the second column (after the file size) represents the permissions set on each item.

### Key Corrections from Your Original Text:

- **The Command:** You mentioned entering `ls -la`, but the screenshot clearly shows you typed `la -ls`. While they produce similar detailed listings, `la` is slightly different, and the `-s` (likely part of `-S` for sorting) specifically orders the list by file size.
- **Hidden Files:** There are **no hidden files** shown in the output. A hidden file in Linux starts with a dot (e.g., `.project_x.txt`). Every file listed in your screenshot starts with a standard character.
- **File Count:** There are **six** other files, not five. You counted: `projects_k.txt`, `projects_m.txt`, `projects_r.txt`, `projects_t.txt`, `projects_x.txt`, and also `vulnerabilities.txt`.

## Describe the permissions string

The 10-character string can be deconstructed to determine who is authorized to access a file or directory, along with their specific permission levels. The positions and what they represent are as follows:

- **1st character:** This character is either a **d** or a hyphen (-) and indicates the file type. A **d** signifies a directory, while a hyphen (-) denotes a regular file.
- **2nd–4th characters:** These characters represent the read (**r**), write (**w**), and execute (**x**) permissions for the **user** (file owner). A hyphen (-) in any of these positions indicates that the corresponding permission is not granted to the owner.
- **5th–7th characters:** These characters represent the read (**r**), write (**w**), and execute (**x**) permissions for the **group**. A hyphen (-) indicates that the permission is not granted to members of the group.
- **8th–10th characters:** These characters represent the read (**r**), write (**w**), and execute (**x**) permissions for **others** (all remaining users on the system). A hyphen (-) indicates that the permission is not granted to other users.

### Example:

Referencing the image, the permissions for **projects\_t.txt** are set to **-rw-rw-r--**:

- **File Type:** The first character is a hyphen (-), indicating that **projects\_t.txt** is a regular file, not a directory.
- **Read Permissions:** The 2nd, 5th, and 8th characters are all **r**, meaning the user, group, and others all have permission to read the file.
- **Write Permissions:** The 3rd and 6th characters are **w**, meaning only the user and group have permission to modify the file.
- **Execute Permissions:** The 4th, 7th, and 10th characters are hyphens (-), meaning no one has permission to execute the file.

## Change file permissions

The organization determined that the "others" user category should not have write permissions for any files within the directory. To comply with this policy, I reviewed the previously listed file permissions and identified that write access needed to be explicitly revoked where applicable.

The command line interaction below demonstrates the remediation process:

```
(swart@Kali)-[~/Projects]
$ chmod o-w projects_k.txt

(swart@Kali)-[~/Projects]
$ ls -la
total 12
drwxr-xr-x  3 swart swart 4096 Jul 30 13:27 .
drwx----- 23 swart swart 4096 Jul 30 13:07 ..
drwxrwxr-x  2 swart swart 4096 Jul 30 13:22 drafts
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_k.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_m.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_r.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_t.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:26 projects_x.txt
-rw-rw-r--  1 swart swart   0 Jul 24 13:59 vulnerabilities.txt

(swart@Kali)-[~/Projects]
$
```

## Explanation of Executed Commands

- **Command Entry:** The screenshot captures two commands executed sequentially: `chmod o-w projects_k.txt` followed by `ls -la`.
- **Permission Modification (`chmod`):** The `chmod` (change mode) utility alters read, write, and execute permissions on Linux files and directories.
  - The first argument, `o-w`, specifies the target category and action—removing (-) write permissions (w) from others (o).
  - The second argument, `projects_k.txt`, specifies the target file.
- **Verification (`ls -la`):** Following the modification, I executed `ls -la` to list all directory contents in long format (including hidden entries like `.` and `..`). The updated output confirms that `projects_k.txt` now reflects permissions of `-rw-rw-r--`, successfully revoking write access for the "others" group while retaining appropriate read and write access for the owner and primary group.

## Change file permissions on a hidden file

The research team recently archived `projects_x.txt`. To maintain data integrity, organization policy dictates that no user should have write access to archived files, while both the user (owner) and group must retain read access.

The command line interaction below demonstrates how I modified the permissions using Linux commands:

```
(swart@Kali)-[~/Projects]
$ chmod u-w,g-w,g+r projects_x.txt

(swart@Kali)-[~/Projects]
$ ls -la
total 12
drwxr-xr-x  3 swart swart 4096 Jul 30 13:27 .
drwx----- 23 swart swart 4096 Jul 30 13:07 ..
drwxrwxr-x  2 swart swart 4096 Jul 30 13:22 drafts
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_k.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_m.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_r.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_t.txt
-r--r--r--  1 swart swart   0 Jul 30 13:26 projects_x.txt
-rw-rw-r--  1 swart swart   0 Jul 24 13:59 vulnerabilities.txt

(swart@Kali)-[~/Projects]
$
```

## Command Breakdown & Verification

- **Command Execution:** The screenshot captures two sequential command entries: the `chmod` modification command followed by `ls -la` for verification.
- **Permission Adjustment (`chmod`):** Using symbolic notation, multiple permission updates were applied in a single command:
  - `u-w`: Removes write permission from the user (owner).
  - `g-w`: Removes write permission from the group.
  - `g+r`: Grants read permission to the group.
  - **Target:** The command was executed specifically on `projects_x.txt`.
- **Output Verification (`ls -la`):** Running `ls -la` lists the detailed directory contents to confirm the changes. The resulting permission string for `projects_x.txt` is now `-r--r--r--`, successfully revoking write permissions across the board while ensuring read-only access for the user, group, and others.

## Change directory permissions

Organization policy specifies that only authorized personnel should have access to traverse the `drafts` directory and view its contents. In Linux file systems, directory traversal requires execute (`x`) permission. To enforce access restrictions, execute permissions must be revoked for all unauthorized entities (such as the group).

The command line interaction below illustrates how I modified the directory permissions using Linux commands:

```
(swart@Kali)-[~/Projects]
$ chmod g-x drafts

(swart@Kali)-[~/Projects]
$ ls -la
total 12
drwxr-xr-x  3 swart swart 4096 Jul 30 13:27 .
drwx----- 23 swart swart 4096 Jul 30 13:07 ..
drwxrw-r-x  2 swart swart 4096 Jul 30 13:22 drafts
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_k.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_m.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_r.txt
-rw-rw-r--  1 swart swart   0 Jul 30 13:27 projects_t.txt
-r--r--r--  1 swart swart   0 Jul 30 13:26 projects_x.txt
-rw-rw-r--  1 swart swart   0 Jul 24 13:59 vulnerabilities.txt

(swart@Kali)-[~/Projects]
$
```

## Output Breakdown & Verification

The terminal screenshot captures the executed `chmod` command followed by `ls -la` to review the updated permissions for all files and directories in long format:

- **Line 1 (.)**: Represents the current working directory (`Projects`).
- **Line 2 (..)**: Represents the parent directory.
- **Line 3 (drafts)**: Displays the targeted directory. Following the execution of `chmod g-x drafts`, the permission string updated to `drwxr-w-r-x` (or `drwxrw-r-x`), showing that execute (`x`) permission was successfully revoked from the group (`g-x`). The owner retains full read, write, and execute permissions (`rw``x`) necessary for managing the directory.
- **Lines 4–9**: Display the remaining regular project files in the directory (`projects_k.txt`, `projects_m.txt`, `projects_r.txt`, `projects_t.txt`, `projects_x.txt`, and `vulnerabilities.txt`).

## Summary

This project focused on establishing robust directory structures, reviewing file system security, and enforcing the principle of least privilege across project assets in a Linux environment (`Kali Linux`). By auditing initial file states and applying targeted permission modifications, access was restricted to safeguard sensitive organizational data and align directory configurations with strict security policies.

## Technical Tasks & Key Accomplishments

### 1. File System Inventory & Permission Auditing

- **Directory Investigation**: Utilized Linux command-line utilities (`ls -la`, `la -ls`) to perform detailed audits of the `Projects` directory, mapping out file ownership, file sizes, and 10-character permission strings across all directories and project files.

- **Permission Analysis:** Deconstructed system permission strings into file type indicators and user, group, and other (u/g/o) permission triads (r/w/x) to identify policy non-compliance and unauthorized access risks.

## 2. Access Control Remediation (chmod)

- **Restricting Unauthorized Write Access (projects\_k.txt):** Enforced policy preventing unauthenticated/other users from modifying organizational data by executing targeted `chmod o-w` commands, securing the file to `-rw-rw-r--`.
- **Archival Data Protection (projects\_x.txt):** Secured archived research files against unauthorized modification while maintaining read availability. Executed multi-attribute modifications (`chmod u-w,g-w,g+r`) to set the file to a read-only state (`-r--r--r--`) for the owner, group, and others.
- **Directory Traversal Restrictions (drafts):** Restricted directory traversal and execution capabilities to authorized project owners. Revoked execute permissions from the group (`chmod g-x drafts`), securing sensitive draft assets while maintaining necessary owner administrative rights.

## 3. Verification & Compliance

- Systematically verified all permission updates post-remediation using long-listing flags to confirm that every file and folder strictly adhered to the organizational security baseline without disrupting necessary workflows.

## Key Skills & Tools Demonstrated

- **Operating System:** Linux (Kali Linux)
- **Command Line Interface (CLI):** `chmod`, `ls`, `mkdir`, `touch`, `rm`, relative & absolute path navigation (`cd`, `~`, `..`)
- **Security Concepts:** Principle of Least Privilege (PoLP), Access Control Lists (ACL) / POSIX Permissions, File System Auditing, Data Integrity Protection